

Article

An Ensemble-Based Machine Learning Approach for Cyber-Attacks Detection in Wireless Sensor Networks

Shereen Ismail , Zakaria El Mrabet and Hassan Reza 

School of Electrical Engineering and Computer Science, University of North Dakota, Grand Forks, ND 58202, USA

* Correspondence: shereen.ismail@ndus.edu

Abstract: Wireless Sensor Networks (WSNs) are the key underlying technology of the Internet of Things (IoT); however, these networks are energy constrained. Security has become a major challenge with the significant increase in deployed sensors, necessitating effective detection and mitigation approaches. Machine learning (ML) is one of the most effective methods for building cyber-attack detection systems. This paper presents a lightweight ensemble-based ML approach, Weighted Score Selector (WSS), for detecting cyber-attacks in WSNs. The proposed approach is implemented using a blend of supervised ML classifiers, in which the most effective classifier is promoted dynamically for the detection process to gain higher detection performance quickly. We compared the performance of the proposed approach to three classical ensemble techniques: Boosting-based, Bagging-based, and Stacking-based. The performance comparison was conducted in terms of accuracy, probability of false alarm, probability of detection, probability of misdetection, model size, processing time, and average prediction time per sample. We applied two independent feature selection techniques. We utilized the simulation-based labeled dataset, WSN-DS, that comprises samples of four internal network-layer Denial of Service attack types: Grayhole, Blackhole, Flooding, and TDMA scheduling, in addition to normal traffic. The simulation revealed promising results for our proposed approach.

Keywords: internet of things; wireless sensor networks; detection; cyber-attacks; security; boosting; bagging; stacking



Citation: Ismail, S.; El Mrabet, Z.; Reza, H. An Ensemble-Based Machine Learning Approach for Cyber-Attacks Detection in Wireless Sensor Networks. *Appl. Sci.* **2023**, *13*, 30. <https://doi.org/10.3390/app13010030>

Academic Editor: Junseop Lee

Received: 1 November 2022

Revised: 13 December 2022

Accepted: 16 December 2022

Published: 20 December 2022



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Wireless Sensor Networks (WSN)s, the key underlying technology that enables the Internet of Things (IoT), have grown rapidly in terms of applications [1]. The tiny, intelligent sensor nodes are typically deployed in the hundreds or thousands to monitor events or physical phenomena. The nodes transmit the collected data to a central node, called the base station (BS), for processing and data fusion [2] or to the IoT cloud for additional investigation and analysis. WSN networks are susceptible to various critical cyber-attacks due to their poor security capabilities and limited node resources [3]. These cyber-attacks have various goals: stealing, altering, hacking data the sensors have collected or flooding the targeted nodes with excess packets in an effort to drain the sensor's batteries and disconnect them from the network, rendering them inoperable and preventing them from sensing or routing traffic [4]. Effective security measures, such as well-defined detection and mitigation techniques, must be in place to address these challenges.

WSNs have unique characteristics that render classical heavyweight security measures, including spread spectrum, cryptography, and key management, insufficient due to constrained resources, such as computational power, data storage, and packet buffering. These detriments have created the need to search for lightweight, effective security mechanisms capable of balancing node resource usage in terms of power, memory, storage, and processing [5–7].

A promising solution to protect WSNs against cyber-attacks is Machine Learning (ML) [8]. Properly trained ML models will allow us to classify normal and malicious

traffic [9]. All traditional supervised learning-based detection methods have flaws related to the learning process, such as dataset specifications in terms of size and number of features, selected ML techniques, and imbalanced classification [10]. Ensemble-based techniques have gained much attention in various fields since they offer better solutions and manage the different trade-offs more effectively than other classical single classifiers. Combining multiple ML algorithms to construct ensemble learners improves prediction capabilities because (1) ensemble approaches involve several ML algorithms cooperating to overcome their respective constraints; (2) the ensemble algorithms can perform averaging or majority voting and learn how to choose the correct classifier; and (3) an ensemble built using the search from a variety of initial points can potentially provide a better approximation to the objective function at hand than any base classifier.

We expected highly accurate classifications that can reduce the chance of false positives and false negatives when detecting different types of network layer WSN-based cyber-attacks, such as false data injection, Sybil, Sinkhole, and Denial of Service (DoS) [11]. Special considerations must be taken when applying these techniques to WSNs due to their unique characteristics. For these reasons, applying ensemble-based ML techniques to detect DoS attacks on WSNs requires further investigation and performance evaluation [12].

We present a lightweight ensemble-based ML approach, Weighted Score Selector (WSS), to detect cyber-attacks targeting WSNs. The proposed approach allows us to choose the most effective model among a set of classifiers to detect attacks. The proposed approach's design is simpler than any other ensemble technique, formed of a pool of conventional supervised ML techniques: Support Vector Machine (SVM), Gaussian Naive Bayes (NB), Decision Tree (DT), K-Nearest Neighbor (KNN), Random Forest (RF), and LightGBM. The model with the best performance in terms of the considered evaluation metrics will be dynamically promoted for the detection process. We investigated the proposed ensemble approach's effectiveness against three classical ensemble techniques: Boosting-based, Bagging-based, and Stacking-based. We conducted performance analysis in terms of accuracy, probability of false alarm, probability of detection, probability of misdetection, processing time, average prediction time per sample, and model size. We applied two independent feature selection techniques to simplify the design. WSN-DS [13] was utilized for the learning process, which contains a sufficient number of samples corresponding to four types of internal DoS network attacks: Grayhole, Blackhole, Flooding, and TDMA scheduling.

The remaining of this paper is structured as follows. Related classical ensemble techniques are reviewed in Section 2. Section 3 described the proposed work methodology. Evaluation results and performance comparisons are discussed in Section 4. Section 5 concludes the paper with future work.

2. Related Work

Ensemble learning is one of the most well-known ML techniques used in the field of cyber-attack classification and detection. These techniques are usually constructed using multiple ML classifiers to solve the same problems and combine the results with one of the voting techniques. Figure 1 depicts the workflow of the three classical ensemble techniques: Boosting, Bagging, and Stacking.

The Boosting ensemble technique process starts by training a set of weak classifiers that learn sequentially, where each classifier provides increasing weights to observations that were poorly predicted by its predecessor until it reaches a terminated condition.

The Bagging ensemble technique is a bootstrap sampling technique used to generate subsets of the dataset. Each subset is used independently to construct a classifier by applying the Decision Tree algorithm. The set of generated classifiers is aggregated through majority voting or averaging to model a Bagging classifier. Bagging has the potential to improve prediction accuracy because it is highly susceptible to any variations in the training data [14].

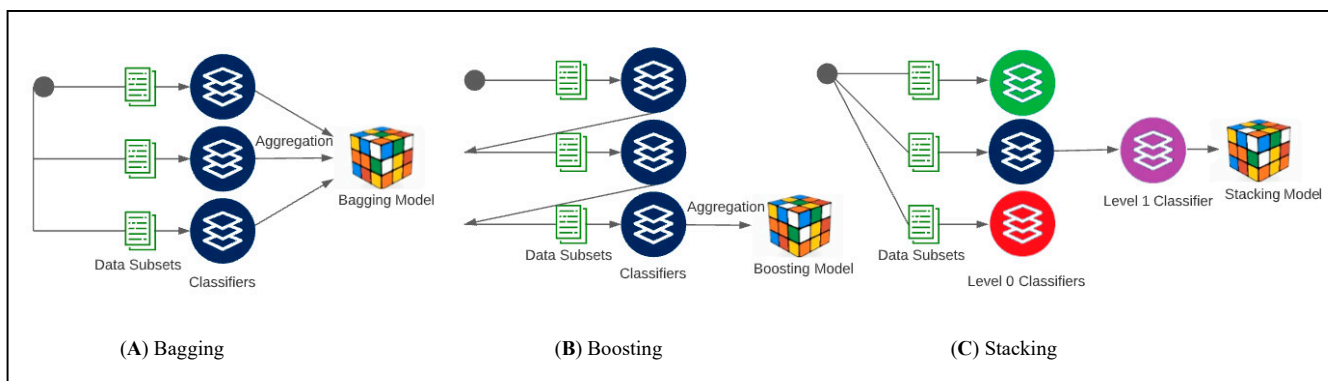


Figure 1. Ensemble techniques: Bagging, Boosting, and Stacking workflow.

Boosting and Bagging combine similar base classifiers, also called homogenous, by averaging or majority voting to build a robust model; however, Stacking-based ensemble learning integrates diverse classification algorithms. This ensemble technique is usually built at two levels: 0 and 1. Multiple base classifiers are considered in the learning process at level 0, while at level 1, one model learns from the best estimate of the level 0 classifiers. Diverse ML techniques at both levels will cause a significant improvement in the classification process and achieve high cyber-attack detection efficiencies.

Several studies have considered ensemble techniques to implement effective intrusion detection systems. For example, the authors in [15] categorized ensemble techniques into homogeneous and heterogeneous. The same base classifiers are used in the classification process for homogeneous ensembles, whereas heterogeneous ensembles consider different base classifiers for the classification process. The authors used the homogeneous ensemble techniques Boosting and Bagging to study and test the NSL-KDD dataset. The authors of [16] used Bagging and Boosting with tree-based algorithms as the base classifier. The proposed models were then examined using two key metrics, accuracy and false alarm rate, then evaluated with the NSL-KDD dataset. The authors of [12] used three ensemble methods, Boosting, Bagging, and Stacking, to increase the efficiency of intrusion detection systems using the UNSW-NB15 and CICIDS2017 datasets. They then compared their results to other classical individual classifiers in the context of IoT smart city applications.

The authors of [17] studied how well the classical Boosting, Bagging, and Stacking ensemble techniques performed in the context of smart grids using the CICDDoS 2019 dataset to detect Distributed-DoS attacks. Their results indicated that the stacking-based technique outperformed the others in terms of accuracy, probability of false alarm, probability of detection, and probability of misclassification. The authors of [18] evaluated the ensemble techniques Bagging, AdaBoost, and Random Forest (RF) with hyperparameter tuning techniques called SMOTE using the imbalanced dataset KDD Cup 99. Their goal was to create an efficient solution for class imbalance and increase intrusion detection classification accuracy.

The authors of [19] proposed an intrusion detection model using the Bagging algorithm based on the information gain ratio to detect Denial of Service (DoS) attacks in a hierarchical clustering WSN. Mutual Information and Kendell's correlation techniques were used for feature selection and 10-fold cross-validation. The Bagging algorithm was used as an ensemble approach to train a set of C4.5 decision trees. The NSL-KDD and WSN-DS datasets were used for other classical techniques to evaluate the proposed model. The authors of [20] proposed an anomaly detection model using the ensemble principle for feature selection in the case of unknown data distribution, lack of labeled data, new attack types, and the existence of several biasing parameters. They evaluated the proposed model using the KDD'99 dataset with detection rate, false alarm rate, and F1-score.

The authors of [21] demonstrated the performance of several ensemble techniques, including Random Forest, Bagging, and Boosting, to detect various types of attacks, such as a DoS using the NSL_KDD dataset. The authors of [22] proposed an ensemble random

forest model for anomaly detection in WSN using a set of base learners (DT, NB, and K-NN) and examined it using a real sensor dataset called AReM. The authors in [23] evaluated homogeneous ensemble models, such as hoeffding adaptive tree (HAT), and adaptive RF and heterogeneous ensemble models built using two base classifiers (adaptive RF and NB) against individual models such KNN, SVM, and NB using the WSN-DS dataset. The authors of [24] implemented a 3-step cascaded multi-layer perceptron (MLP)-based ensemble learning model for intrusion detection using the KDD Cup99 and NSL-KDD datasets. In [25], the authors proposed an ensemble method using RF, Density- Based Spatial Clustering of Applications with Noise (DBSCAN) and Restricted Boltzmann Machine (RBM) as base classifiers and using two types of Bayesian algorithm as a combination methods for intrusion detection in WSN and using NSL-KDD dataset.

These studies presented the use of ensemble techniques for efficient intrusion detection systems; however, most ensembles developed to detect possible vulnerabilities rely on only one type of base-level classifier. These techniques have a deeper and more complex structure than single conventional ML classifiers because of the multiple ML-based classification methods have been integrated; therefore, ensemble ML techniques must be investigated further to evaluate their performance since WSNs have unique specifications. We propose using WSS, a lightweight ensemble-based ML approach, to detect DoS attacks on WSNs. Our proposed approach comprises a set of individual supervised ML classifiers that are evaluated based on several evaluation metrics, in which one classifier will be identified and selected as the most effective classifier to detect cyber-attacks to gain higher detection performance quickly. The proposed approach is designed to be simpler than other well-known ensemble techniques since there is no integration of the outcomes of the base ML models, it is diverse in terms of selecting the base classifiers, and flexible in adopting any number of ML models and any set of evaluation metrics. We can prioritize certain metrics with higher weights, or scores, if they have higher importance based on the learning conditions and application domain with our proposed approach. A similar approach for classifying and detecting DoS attacks on WSNs, considering their special requirements for the different ML learning workflow phases, has not been presented in previous studies to the best of the author's knowledge; however, a similar concept was presented in [26] to dynamically select the best classifier to detect GPS spoofing attacks targeting unmanned aerial vehicles.

3. Methodology

The conceptual diagram for any ML learning methodology's workflow can be divided into several phases: dataset selection, data preprocessing, feature selection, ML model training, testing and validation, and hyperparameter tuning (Figure 2). We have created several balanced datasets with a mixture of normal and attack instances for binary classification to reduce the computational and space overhead in the learning process. This approach will allow us to implement more effective classification learning models that meet WSN constraints. The detection model for binary classification identifies each incoming data sample as attack or normal [27].

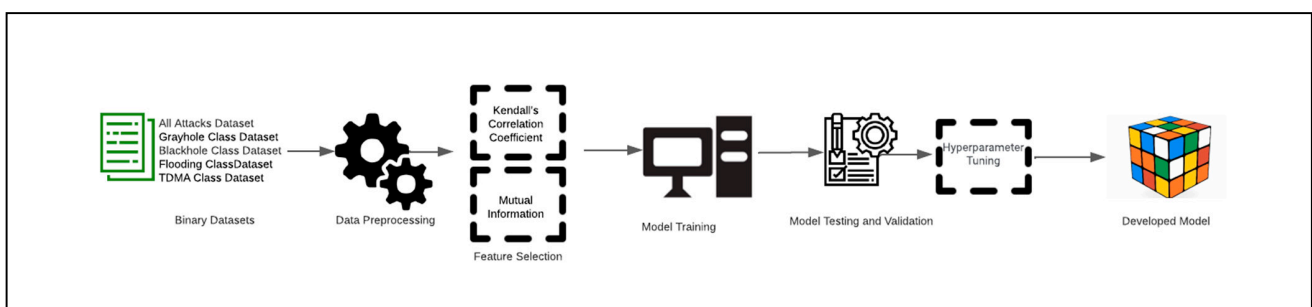


Figure 2. Conceptual diagram for the proposed ML learning methodology's workflow.

3.1. Dataset Selection and Data Preprocessing

We used WSN-DS [13], a specialized simulation-based dataset collected for hierarchically cluster-based WSNs in this study. Network-layer attacks, including selective forwarding, spoofing, blackhole, wormhole, Sybil, sinkhole, hello flood, and altered or replayed routing information, are the most common attacks targeting WSNs. Four types of inside network-layer DoS attacks, Grayhole, Blackhole, Flooding, and TDMA scheduling, were used based on the employed dataset. An attack is called Grayhole when malicious cluster heads (CHs) selectively drop some packets before forwarding messages to the BS. The malicious CHs discard every packet that has to be forwarded to the BS during a Blackhole attack. A flooding attack occurs when a malicious node falsely identifies itself as a CH and repeatedly sends signals that drain the other sensors' power. Another attack type, TDMA scheduling, occurs when a malicious CH assigns multiple cluster members to deliver data in the same time slot, resulting in packet collision and data loss.

Several dataset specifications, such as high variation in class types and sizes, dataset dimensionality, and class imbalance, can degrade the ML model's learning process performance. We extracted several balanced datasets for our experiments due to this problem. Table 1 lists the specifications of the original imbalanced WSN-DS dataset and the extracted balanced dataset. Table 2 presents a set of single-class datasets for each considered attack.

Table 1. Original and extracted balanced datasets.

Dataset	Class	Size	Proportion (%)	Total
Original Imbalanced Dataset	Normal	340,066	90.77	374,661
	Grayhole	14,596	3.90	
	Blackhole	10,049	2.68	
	Flooding	6638	1.77	
	Scheduling	3312	0.88	
Extracted Balanced Dataset	Normal	13,000	50	26,000
	Grayhole	3250	12.5	
	Blackhole	3250	12.5	
	Flooding	3250	12.5	
	Scheduling	3250	12.5	

Table 2. Extracted single-class datasets.

Dataset	Class Type	Size	Proportion (%)	Total Size
Grayhole Dataset	Normal	3250	50	6500
	Grayhole	3250	50	
Blackhole Dataset	Normal	3250	50	6500
	Blackhole	3250	50	
Flooding Dataset	Normal	3250	50	6500
	Flooding	3250	50	
TDMA Dataset	Normal	3250	50	6500
	TDMA	3250	50	

The balanced dataset version, which extracts four different types of attacks, consists of 26,000 samples, with 3250 samples for each attack type. A balanced dataset prevents classifiers from being biased toward the majority class [28]. The dataset size was also reduced to approximately 10% of the original dataset, which is preferred in the context of learning models for WSNs to lower computation and memory needs. We looked at the sample size for each type of attack in the original dataset to determine if the TDMA Scheduling size represented the minimum number of classes, then used that as a base number for individual attack datasets, for an approximate number of 3250 samples per dataset for each type of attack.

3.2. Feature Selection

Nineteen features comprised the associated dataset, including class type (Table 3). Any detection model's effectiveness depends on the choice of pertinent features in addition to the selected learning algorithm [19]. Appropriate feature selection criteria are necessary to extract an optimal subset of features, decreasing the dataset's dimensionality and increasing learning process speed [29].

Table 3. Key Features of Dataset [13].

Feature	Description
Time	Simulation time at the moment for the sensor node
ID	Unique number to identify the node during the simulation period
Is_CH	Flag bit to determine whether the node is CH or normal
Who CH	Unique number to identify the CH of the current round
Dist_To_CH	The current round's distance between the node and its CH
ADV_S	Number of advertised broadcast messages sent to the nodes by the CH
ADV_R	Number of advertised broadcast messages received from the CHs by nodes
JOIN_S	Number of join request messages sent by the nodes to the CH
JOIN_R	Number of join request messages sent from the nodes by the CH
DATA_S	Number of data packets sent to the CH from the node
DATA_R	Number of data packets received from the CH from the node
Data_Sent_To_BS	Number of data packets sent toward the BS
dist_CH_To_BS	Distance between the CH and the BS
SCH_S	Number of advertised TDMA schedule broadcast messages sent to the nodes
SCH_R	Number of advertised TDMA scheduled messages received from the CHs
Rank	Order of the node in the TDMA schedule
send_code	A special code for data exchange
Consumed Energy	During the previous round, the estimated energy used
Attack type	Grayhole, Blackhole, Flooding, Scheduling

We applied two independent feature selection techniques: Kendall's correlation coefficient and Mutual Information (MI). The authors of [30] investigated the Kendall's correlation coefficient and MI approaches, establishing that these two approaches together can determine the optimal feature subset and reduce dataset dimensionality. Kendall's correlation coefficient and MI techniques measure the correlation between attributes to generate optimal feature subsets. Kendall's correlation is one of the most commonly used measures with ordinal or interval data; however, it works best with data that has at least one ordinal variable.

MI is a statistical independence measure that estimates the dependence among attributes. This measure has two essential characteristics: it measures any kind of relationship, or dependencies, between random variables, and it is invariant under invertible and differentiable feature space transformations, such as translations and rotations, as well as any transformation that preserves the initial order of the feature vector's component elements [31]. MI can be calculated according to the following equation [30] to determine the amount of reduced uncertainty in one attribute X if the other attribute B is given.

$$MI(X, Y) = \sum_{i=1}^r \sum_{j=1}^t P(X_i, Y_j) * \log \left(\frac{P(X_i, Y_j)}{P(X_i) * P(Y_j)} \right)$$

MI, one of the filter techniques, is computationally less expensive when compared to wrapper and embedded techniques, making it more appropriate in the context of WSNs. *MI* evaluates the relevance between each feature and class label; therefore, the feature with a high *MI* value means that this feature has high predictive power.

3.3. Proposed Ensemble Machine Learning Approaches

We present a lightweight ensemble-based ML approach, WSS, to detect cyber-attacks on WSNs using a set of fundamentally diverse single classifiers in the ensemble. Multiple classifiers have been trained, tested, and evaluated using several evaluation metrics to differentiate the most effective classifier for detecting attacks. This proposed approach was built using a pool of heterogeneous supervised ML techniques: SVM, NB, DT, KNN, RF, and LightGBM. We assumed that the individual models used to build a successful WSS were diverse to create a heterogeneous ensemble. Heterogeneous ensembles comprised of different classical individual classifiers were used to increase diversity while maintaining the default settings of the classifiers' parameters during the learning process.

We also compared WSS performance to the three classical ensemble techniques: Bagging-based, Boosting-based, and Stacking-based. The Bagging-based model was built using a decision tree algorithm, the Boosting-based model was implemented using an AdaBoost algorithm, and the Stacking-based model was implemented using three different classification algorithms for level 0: KNN, RF, and NB classifiers. Logistic Regression (LR) was used as a level 1 classifier. We aimed to investigate how these ensemble techniques can be applied to detect attacks targeting WSNs.

Weak classifiers were fitted iteratively to the training data for the Boosting-based algorithm [10]. The classification was boosted gradually by emphasizing samples that were modeled poorly by the weak classifiers and iteratively updating their weights each round to create a single robust classifier. AdaBoost minimizes exponential loss according to the following equation [32]:

$$F_T(x) = \sum_{t=1}^T f_t(x)$$

where f_t represents each weak classifier for an object x

$$E_t = \sum_i E[F_{t-1}(x_i) + a_t h(x_i)]$$

where each weak classifier makes its hypothesis $h(x_i)$ with a coefficient a_t to minimize the sum of the weighted error rate.

The Bagging-based algorithm, or bootstrap aggregation, builds a group of decision trees as base classifiers using multiple versions of the training dataset. Each decision tree is individually classified as a weak learner, then the trees are combined by voting to construct a strong learner as a single model. This process is performed repeatedly for individual trees in the ensemble when a new instance must be classified. Each tree outputs a vote for a class [33]. The class with the maximum number of votes determines the final prediction for each new instance.

The Stacking algorithm selects the best classifier based on combining predictions from different individual classifiers to improve stacked generalization. Implementing the stacking classifier involves two types of models: (1) base models (level 0): KNN, RF, and NB classifiers, and (2) meta-model (level 1): LR classifier. The key concept behind the Stacking algorithm lies in using the meta classifier that learns from the level 0 classifiers to predict data samples.

The primary difference between WSS and other ensemble classifiers is that WSS final prediction is promoted by one classifier that achieves higher performance compared to others in terms of the considered metrics: there is no integration of the outcomes of the base models. The prediction result is computed by combining the results of all base classifiers

using averaging or majority voting in other ensemble techniques. Figure 3 depicts the general overview of the presented approach: WSS. We used the same dataset during the learning process for the set of base classifiers. The base classifiers were trained, tested, and evaluated using the following list of evaluation metrics: accuracy (Acc), probability of false alarm or false positive rate (P_{fa}), probability of detection or true positive rate (P_d), probability of misdetection or false negative rate (P_{md}), and positive predictive value (PPV) in addition to processing time (PT), average prediction time per sample (TT), and model size (Mem) (Figure 3). Each evaluation metric was assigned a weight, called a score. These scores can be equal or varied according to each metric's importance in the evaluation and validation process. Higher-importance metrics are assigned higher scores. The algebraic sum of the scores was calculated for each base classifier, where that classifier achieves the best score for that metric. The best single classifier among the pool of base classifiers is the one that achieves the highest algebraic sum of scores and will be selected to detect any upcoming attacks.

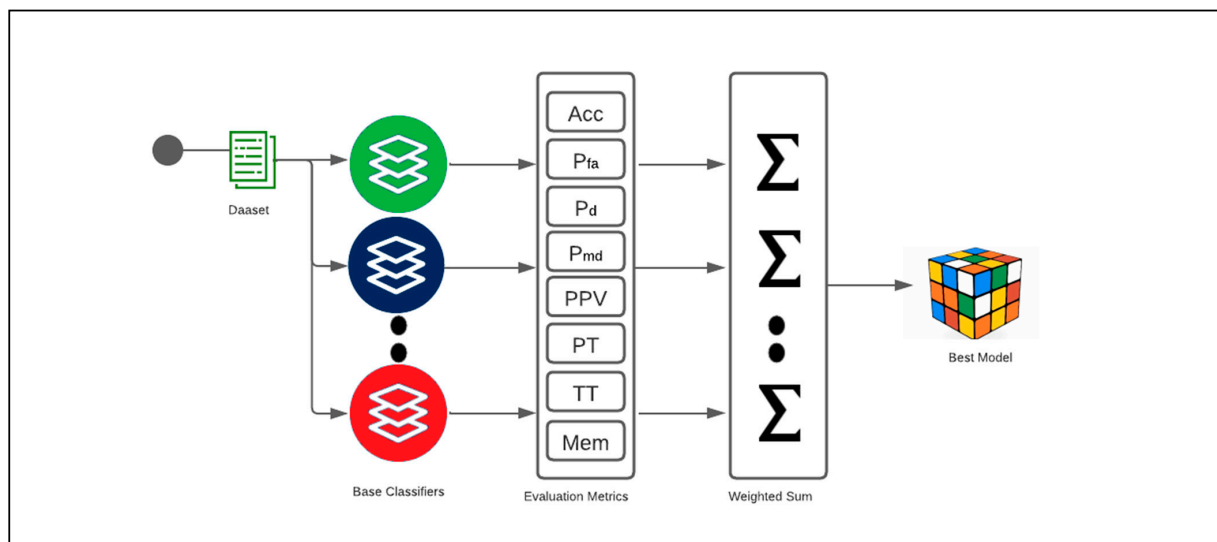


Figure 3. Workflow for the proposed approach, WSS.

The concept of assigning a different score for each evaluation metric depends on the performance trade-off and baseline standardization. Several studies have considered different evaluation metrics for measuring the performance of the ML techniques used for assessing their proposed models' ability to detect cyber-attacks on WSNs; however, they did not emphasize any specific metric. We have considered the most common and widely used metrics, Acc, P_d , P_{fa} , P_{md} , and PPV, in addition to TT, PT, and Mem, which also matter in the context of WSNs for this reason.

We considered base classifiers $C = \{C_1, C_2, \dots, C_i\}$, evaluation metrics $M = \{M_1, M_2, \dots, M_j\}$, and scores $w_j = \{w_1, w_2, \dots, w_j\}$ for the WSS approach, where i is the index of the classifiers and j is the index of the evaluation metrics. We initially calculated the evaluation metrics for every base classifier C_i to have a matrix of $M(i, j)$, then we determined the sum ($w(i, j)$) for every C_i with the best $M(i, j)$. We supposed that the best value for every $M(i, j)$ in terms of Acc, P_d , and PPV is the maximum, while in terms of P_{fa} , P_{md} , TT, PT, and Mem is the minimum. The model achieving the max (sum ($w(i, j)$)) will be the one selected for the final detection of any upcoming traffic. The proposed WSS approach will then be implemented to distinguish the model with the greatest number of metrics that have the best values using Max (sum ($w(i, j)$)).

Our proposed approach does not need extensive processing or sophisticated hardware to be implemented compared to other approaches; therefore, we can assume it does not add any computational complexity to the selected base classifiers. This approach's effectiveness depends highly on the performance of the selected set of base classifiers, where the

optimal model elected at the last stage will have the best scores in terms of the evaluation metrics used.

We differentiated between high and low-importance metrics for the score value w_j assigned to each of the evaluation metrics, M_j . We assumed that accuracy, probability of misdetection, processing time, and prediction time were high-importance metrics, while the probability of detection, probability of false alarm, and model size in memory were considered low-importance metrics. High-importance metrics were assigned higher score values compared to low-importance metrics; therefore, we used weights of $w_{Acc} = 0.2$, $w_{Pd} = 0.1$, $w_{Pfa} = 0.1$, $w_{Pmd} = 0.15$, $w_{Processing\ T} = 0.2$, $w_{Prediction\ T} = 0.15$, and $w_{PPV} = 0.1$. Then, $\text{sum}(w(i, j))$ is calculated for every model M_j , looking for $\text{Max}(\text{sum}(w(i, j)))$ to be the selected model with the best-weighted score summation and chosen as the promoted one to perform the detection of any incoming signal. In case two classifiers obtain the same calculated $\text{sum}(w(i, j))$, we assume arbitrarily that any of the two classifiers is selected as the active classifier for the detection. It is worth mentioning that the weights assigned to each metric can be dynamically adjusted according to the learning conditions and the application domain.

4. Simulation Results

The parameters Acc , P_d , P_{fa} , P_{md} , and PPV, then TT, PT, and Mem, were used to evaluate and compare WSS performance to the three classical ensemble approaches. Equations (1)–(5) formulae were used to calculate the first five metrics:

$$P_d = \frac{T_p}{T_p + F_N} \times 100 \quad (1)$$

$$P_{fa} = \frac{F_p}{T_N + F_p} \times 100 \quad (2)$$

$$P_{md} = \frac{F_N}{T_p + F_N} \times 100 \quad (3)$$

$$PPV = \frac{T_p}{T_p + F_p} \times 100 \quad (4)$$

$$Acc = \frac{T_p + T_N}{T_p + T_N + F_p + F_N} \times 100 \quad (5)$$

where the true positive, true negative, false positive, and false negative values, respectively, are denoted by TP, TN, FP, and FN. These attributes were derived from the training and testing datasets and computed from the confusion matrix. Mem was considered by assessing each model's size as an entity that must be kept in memory until it is time to run. PT is largely influenced by the size of the training dataset and the ML technique; it can be defined as the amount of time needed to train and test each model. TT estimates the time required to determine if the present sample is normal or an attack.

A 3-Fold cross-validation technique was applied, which uses every sample to create training and testing subsets from the corresponding dataset. We deleted two unnecessary features from the dataset, the Time and ID columns, before applying two independent feature selection methods to find the correlated and low-importance features. Figure 4 depicts the results of applying Kendall's correlation and MI to the extracted balanced dataset. Table 4 lists the features removed from the extracted balanced dataset in addition to the four single-class datasets after applying the two feature selection techniques. MI represents the dependencies of the features whose values range from 0 to 1. High MI values represent important features, while low MI values are features with lower importance. The characteristics we eliminated from the dataset are the features with MI values less than 0.1. Two features from the balanced all_attacks dataset needed to be removed: Data_Sent_To_BS and dist_CH_To_BS since they had MI values less than 0.1. We also had to remove SCH_S, JOIN_R, and DATA_R from the balanced dataset retrieved for Grayhole,

and the Data_Sent_To_BS, dist_CH_To_BS, SCH_S, DATA_R, and JOIN_R features needed to be removed from the Blackhole dataset. The JOIN_R, SCH_S, and DATA_R features were removed from the Flooding dataset, and Data_Sent_To_BS, dist_CH_To_BS, ADV_R, and DATA_R features were removed from the TDMA dataset during the feature selection phase before the learning process was started.

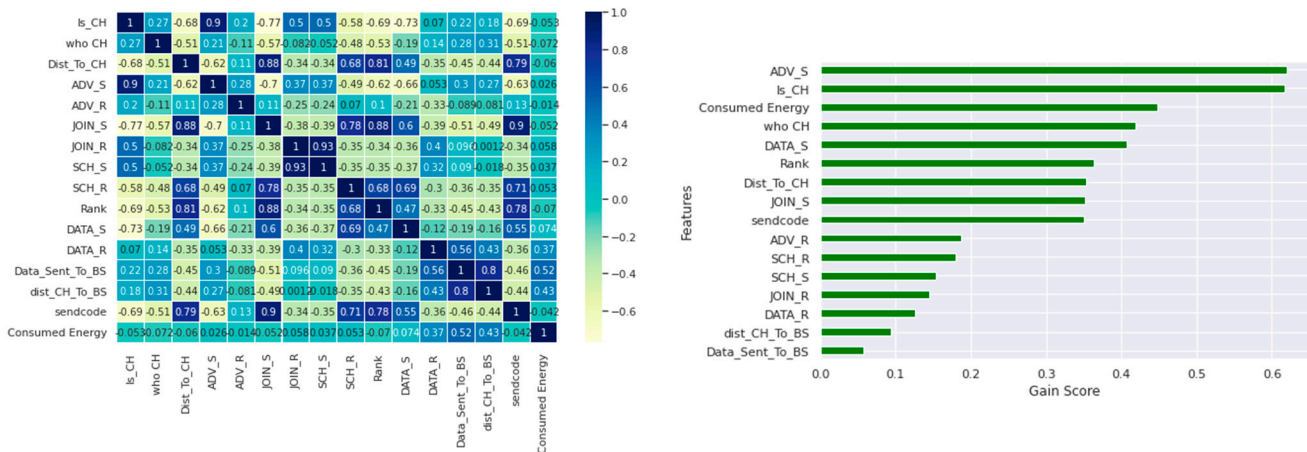


Figure 4. Feature selection results for the extracted balanced dataset.

Table 4. The Results of Mutual Information.

Dataset	Removed Features
All_Attacks Dataset	Data_Sent_To_BS, dist_CH_To_BS
Grayhole Class Dataset	SCH_S, JOIN_R, DATA_R
Blackhole Class Dataset	Data_Sent_To_BS, dist_CH_To_BS SCH_S, DATA_R, JOIN_R
Flooding Class Dataset	JOIN_R, SCH_S, DATA_R
Scheduling Class Dataset	dist_CH_To_BS, Data_Sent_To_BS, ADV_R, DATA_R

We investigated the results of each of the evaluation metrics individually before starting our discussion on the simulation results for the WSS model vs. the Boosting, Bagging, and Stacking models. One of our observations was related to the model size for each considered approach. The model size is represented by how many bytes the classifier reserves in memory after the learning process is finished and once it is ready to be deployed on the sensor nodes.

We noticed that the model sizes for all base classifiers in the WSS were 23, 25, 23, 22, 32, and 28 Bytes for SVC, NB, KNN, DT, RF, and LightGBM, respectively while the other three ensemble model sizes were 27, 28, and 28 Bytes for Bagging, Stacking, and Boosting, respectively. The model sizes were very similar; therefore, we did not need to include the model size with the evaluation metrics in the WSS implementation. We also excluded model size from the comparison between the proposed approach and the three ensemble techniques.

Figure 5 illustrates the simulation results for the proposed approach, WSS, as well as the Boosting, Bagging, and Stacking techniques. In terms of accuracy, WSS achieved the best results considering the All_attacks, Grayhole, and Flooding datasets with accuracies of 99.3%, 98.95%, and 100%, respectively (Figure 5A). WSS, Bagging, Boosting, and Stacking achieved an accuracy of 99% for the Blackhole dataset. The best TDMA dataset results were from Boosting and Stacking, with an accuracy of approximately 100%, while the accuracy for Bagging and WSS was 99.92%.



Figure 5. Results of WSS, Bagging, Boosting, and Stacking in terms of the evaluation metrics.

WSS, Bagging, Stacking, and Boosting classifiers achieved approximately the same results regarding probability of detection (Figure 5B). Boosting was higher for the Blackhole dataset, at 99.69%, and Stacking was higher for the Grayhole dataset, at 99.08%. All models using the Flooding and TDMA datasets achieved the same results, 100%.

Figure 5C displays the results for probability of false alarm. WSS achieved the best results for the All_attacks, Blackhole, Grayhole, Flooding, and TDMA datasets, at 1%, 1.23%, 1.54%, 0%, and 0.155%, respectively. Boosting achieved the same result as WSS for the Flooding Dataset, while Bagging achieved the same result as WSS for the TDMA dataset.

WSS achieved the best results for probability of misdetection: 0.2% and 0.76% for the All_attack and Grayhole datasets (Figure 5D). Stacking and Boosting achieved the best result for the Blackhole dataset: 0.31%. WSS, Bagging, Stacking, and Boosting all achieved 0% for the Flooding and TDMA datasets.

WSS performed the best in terms of positive predictive rate for the All_attacks and Blackhole datasets, at 98.95% and 98.78%, respectively (Figure 5E). WSS and Boosting had the best result for the Grayhole dataset: with 98.48%, while WSS, Stacking, and Boosting achieved the best result of approximately 100% for the Flooding dataset. WSS and Bagging had the best result for the TDMA dataset: 99.85%.

Bagging achieved the best results for the All_attack, Grayhole, and Flooding datasets with 0.252, 0.09, and 0.108 s in terms of processing time, while WSS had the best results for the Blackhole and TDMA datasets, at 0.015 and 0.0165 s, respectively (Figure 5F). Stacking had the worst results for all datasets in terms of processing time: 6.22, 2.95, 2.03, 3.27, and 3.29 s for the All_attacks, Blackhole, Grayhole, Flooding, and TDMA datasets, respectively. We argue that the complexity introduced by the Stacking ensemble structure is one of the reasons that influences its processing time.

Bagging had the best results for average prediction per sample with the All_attacks, Grayhole, and Flooding datasets, at 2.025, 3.11, and 3.56 μ seconds, respectively, followed by WSS at 3.277, 4.46, and 5.35 μ seconds, respectively (Figure 5G). WSS had the best prediction time per sample for the Blackhole and TDMA datasets, at 1.19 and 1.33 μ seconds, respectively.

This paper's key findings can be summarized as follows:

- The original dataset was divided into multiple binary balanced datasets to simplify the classifications and reduce the cost of computation and memory before the data preprocessing phase to avoid multi-class issues.
- Two independent feature selection techniques, Kendall's correlation coefficient, and MI, can be used to determine feature-reduced datasets. Kendall's correlation works best with ordinal variables, and Mutual Information measures the relevance between each individual feature and the class labels so that the features with the highest predictive ability have larger mutual information values. Reducing the number of features minimizes the model's learning time substantially and improves the classifier's performance.
- We used three classical ensemble classifiers: Bagging, Boosting, and Stacking. The Bagging-based model was built based on a decision tree algorithm, the Boosting-based model was built with the AdaBoost algorithm, and the Stacking-based model was implemented using three different classification methods for level 0, KNN, RF, NB, and LR for the level 1 classifier.
- The proposed model, WSS, achieved comparable results to other classical ensemble classifiers in terms of accuracy, probability of false alarm, probability of detection, probability of misdetection, positive predictive rate, processing time, and average prediction time per sample. Further analysis and testing are required to prove its effectiveness for other attack types in energy-constrained networks.
- Ensemble techniques functioned well when detecting cyber-attacks targeting WSN for most of the results and achieved similar values, except for the Stacking algorithm. Bagging has the best processing time and average prediction time per sample, while Stacking performed the worst in terms of processing time and prediction time per sample, making it a poor choice to detect DoS attacks on a WSN.

- A comprehensive analysis of the results indicates that WSS, along with the other ensemble techniques, performs well when detecting most DoS attacks in terms of accuracy, probability of false alarm, probability of detection, and probability of misdetection with the default hyperparameter tuning settings. WSS, followed by Bagging, are the best candidates for detecting WSN attacks using the labeled datasets extracted from WSN-DS since both achieve reasonable results regarding the evaluation metrics used in this study. Further investigation and analysis using other datasets are required considering other types of attacks targeting WSNs.
- WSNs have unique properties caused by constrained power, storage, and bandwidth; therefore, the proposed approach, WSS, achieved promising results for identifying cyber-attacks that target WSNs. The proposed approach is simpler, diverse in terms of selecting base classifiers, and flexible in adopting any number of ML models compared to Bagging, Boosting, and Stacking ensemble techniques. The proposed approach selects the appropriate ML model dynamically, which could significantly improve detection performance.

5. Conclusions and Future Work

Wireless Sensor Networks, a key underlying technology of the Internet of Things, are prone to several types of cyber-attacks that could compromise availability, privacy, control, and reliability. One of the effective methods for detecting and mitigating cyber-attacks on WSNs is Machine Learning. Classifier ensembles have been successfully used to detect cyber-attacks in various scenarios. We proposed a new ensemble-based approach, Weighted Score Selector, which uses a pool of conventional supervised ML techniques: Support Vector Machine, Gaussian Naive Bayes, Decision Tree, K-Nearest Neighbor, Random Forest, and LightGBM. The proposed approach's performance was compared to three classical ensemble techniques that integrate the decisions of multiple machine learning models to detect cyber-attacks on WSNs. Performance comparisons were conducted using the following metrics: accuracy, probability of false alarm, probability of detection, probability of misdetection, memory usage, processing time, and prediction time per sample.

Evaluation results indicated that the proposed ensemble approach yielded promising results in terms of probability of detection, probability of false positives, and probability of misdetection. Future work could entail expanding the presented approach to examine more evaluation metrics or additional heterogeneous classifiers using other datasets. This work can be accomplished by extending the pool of base classifiers to include other machine learning algorithms, such as unsupervised and reinforcement learning, and activating a subset of them to contribute to the detection process, depending on the learning conditions and application domain. The activated base classifiers will be selected to match the requirements of the environment where they will be applied.

Author Contributions: Conceptualization, S.I. and Z.E.M.; Software, S.I.; Data curation, S.I.; Formal Analysis, S.I. and Z.E.M.; Investigation, S.I.; Methodology, S.I. and Z.E.M.; visualization, S.I.; Writing original draft, S.I. and H.R.; review & editing, S.I. and H.R.; Project administration, S.I. and H.R.; Validation, S.I. and H.R.; Supervision, H.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Dataset is available on GitHub and other websites.

Acknowledgments: The authors acknowledge the support of the University of North Dakota College of Engineering & Mines.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Ismail, S.; Alkhader, E.; Ahmad, A. Prison perimeter surveillance system using WSN. *J. Comput. Sci.* **2017**, *13*, 674–679. [\[CrossRef\]](#)
2. Ismail, S.; Alkhader, E.; Elnaffar, S. Object tracking in Wireless Sensor Networks: Challenges and solutions. *J. Comput. Sci.* **2016**, *12*, 201–212. [\[CrossRef\]](#)
3. Ahmad, R.; Wazirali, R.; Abu-Ain, T. Machine Learning for Wireless Sensor Networks Security: An Overview of Challenges and Issues. *Sensors* **2022**, *22*, 4730. [\[CrossRef\]](#) [\[PubMed\]](#)
4. Elhoseny, M.; Hassanien, A.E. Secure data transmission in WSN: An overview. *Stud. Syst. Decis. Control* **2019**, *165*, 115–143. [\[CrossRef\]](#)
5. Ismail, S.; Khoei, T.T.; Marsh, R.; Kaabouch, N. A Comparative Study of Machine Learning Models for Cyber-attacks Detection in Wireless Sensor Networks. In Proceedings of the 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), New York, NY, USA, 1–4 December 2021; pp. 313–318.
6. Ismail, S.; Dawoud, D.; Reza, H. A Lightweight Multilayer Machine Learning Detection System for Cyber-attacks in WSN. In Proceedings of the 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 26–29 January 2022; pp. 481–486.
7. Ismail, S.; Reza, H. Evaluation of Naïve Bayesian Algorithms for Cyber-Attacks Detection in Wireless Sensor Networks. In Proceedings of the 2022 IEEE World AI IoT Congress (AIIoT), Seattle, WA, USA, 6–9 June 2022; pp. 283–289.
8. Poornima, I.G.A.; Paramasivan, B. Anomaly detection in wireless sensor network using machine learning algorithm. *Comput. Commun.* **2020**, *151*, 331–337. [\[CrossRef\]](#)
9. Ahmad, Z.; Shahid Khan, A.; Wai Shiang, C.; Abdullah, J.; Ahmad, F. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans. Emerg. Telecommun. Technol.* **2021**, *32*, e4150. [\[CrossRef\]](#)
10. Yang, Z.; Xu, L.; Cai, Z.; Xu, Z. Knowledge-Based Systems Re-scale AdaBoost for attack detection in collaborative filtering recommender systems. *Knowl.-Based Syst.* **2016**, *100*, 74–88. [\[CrossRef\]](#)
11. Saba, T. Intrusion Detection in Smart City Hospitals using Ensemble Classifiers. In Proceedings of the 2020 13th International Conference on Developments in eSystems Engineering (DeSE), Liverpool, UK, 14–17 December 2020; pp. 418–422.
12. Rashid, M.; Kamruzzaman, J.; Hassan, M.M. Cyberattacks Detection in IoT-Based Smart City Applications Using Machine Learning Techniques. *Int. J. Environ. Res. Public Health* **2020**, *17*, 9347. [\[CrossRef\]](#)
13. Almomani, I.; Al-Kasasbeh, B.; AL-Akhras, M. WSN-DS: A Dataset for Intrusion Detection Systems in Wireless Sensor Networks. *J. Sens.* **2016**, *2016*, 4731953. [\[CrossRef\]](#)
14. Dou, J.; Yunus, A.P.; Bui, D.T.; Merghadi, A.; Sahana, M.; Zhu, Z.; Chen, C.-W.; Han, Z.; Pham, B.T. Improved landslide assessment using support vector machine with bagging, boosting, and stacking ensemble machine learning framework in a mountainous watershed, Japan. *Landslides* **2020**, *17*, 641–658. [\[CrossRef\]](#)
15. Masoodi, F.S.; Abrar, I.; Bamhdi, A.M.; Arabia, S. An effective intrusion detection system using homogeneous ensemble techniques. *Int. J. Inf. Secur. Priv.* **2021**, *16*, 1–18. [\[CrossRef\]](#)
16. Pham, N.T.; Foo, E.; Suriadi, S.; Jeffrey, H.; Lahza, H.F.M. Improving performance of intrusion detection system using ensemble methods and feature selection. In Proceedings of the Australasian Computer Science Week Multiconference, Brisbane, Australia, 29 January–2 February 2018. [\[CrossRef\]](#)
17. Khoei, T.T.; Aissou, G.; Hu, W.C.; Kaabouch, N. Ensemble Learning Methods for Anomaly Intrusion Detection System in Smart Grid. In Proceedings of the 2021 IEEE International Conference on Electro Information Technology (EIT), Mt. Pleasant, MI, USA, 14–15 May 2021; pp. 129–135. [\[CrossRef\]](#)
18. Tan, X.; Su, S.; Huang, Z.; Guo, X.; Zuo, Z.; Sun, X.; Li, L. Wireless sensor networks intrusion detection based on SMOTE and the random forest algorithm. *Sensors* **2019**, *19*, 203. [\[CrossRef\]](#) [\[PubMed\]](#)
19. Dong, R.-H.; Yan, H.-H.; Zhang, Q.-Y. An Intrusion Detection Model for Wireless Sensor Network Based on Information Gain Ratio and Bagging Algorithm. *Int. J. Netw. Secur.* **2020**, *22*, 218–230.
20. Garg, S.; Singh, A.; Batra, S.; Kumar, N.; Obaidat, M.S. EnClass: Ensemble-based classification model for network anomaly detection in massive datasets. In Proceedings of the GLOBECOM 2017—2017 IEEE Global Communications Conference, Singapore, 4–8 December 2017. [\[CrossRef\]](#)
21. Abdulla, N.N.; Hasoun, R.K. Homogenous Ensemble Learning for Denial of Service Attack Detection. *J. Algebr. Stat.* **2022**, *13*, 355–366.
22. Biswas, P.; Samanta, T. Anomaly detection using ensemble random forest in wireless sensor network. *Int. J. Inf. Technol.* **2021**, *13*, 2043–2052. [\[CrossRef\]](#)
23. Tabbaa, H.; Ifzarne, S.; Hafidi, I. An Online Ensemble Learning Model for Detecting Attacks in Wireless Sensor Networks. *arXiv* **2022**, arXiv:2204.13814.
24. Sarkar, A.; Sharma, H.S.; Singh, M.M. A supervised machine learning-based solution for efficient network intrusion detection using ensemble learning based on hyperparameter optimization. *Int. J. Inf. Technol.* **2022**, *2022*, 1–12. [\[CrossRef\]](#)
25. Otoum, S.; Kantarci, B.; Mouftah, H.T. A Novel Ensemble Method for Advanced Intrusion Detection in Wireless Sensor Networks. In Proceedings of the ICC 2020—2020 IEEE International Conference on Communications (ICC), Dublin, Ireland, 7–11 June 2020. [\[CrossRef\]](#)
26. Khoei, T.T.; Ismail, S.; Kaabouch, N. Dynamic Selection Techniques for Detecting GPS Spoofing Attacks on UAVs. *Sensors* **2022**, *22*, 662. [\[CrossRef\]](#)

27. Iwendi, C.; Khan, S.; Anajemba, J.H.; Mittal, M.; Alenezi, M.; Alazab, M. The Use of Ensemble Models for Multiple Class and Binary Class Classification for Improving Intrusion Detection Systems. *Sensors* **2020**, *20*, 2559. [\[CrossRef\]](#)
28. Karatas, G. Increasing the Performance of Machine Learning-Based IDSs on an Imbalanced and Up-to-Date Dataset. *IEEE Access* **2020**, *8*, 32150–32162. [\[CrossRef\]](#)
29. Jafari, F.; Dorafshan, S. Bridge Inspection and Defect Recognition with Using Impact Echo Data, Probability, and Naive Bayes Classifiers. *Infrastructures* **2021**, *6*, 132. [\[CrossRef\]](#)
30. Dubey, G.P.; Bhujade, R.K. Materials Today: Proceedings Optimal feature selection for machine learning based intrusion detection system by exploiting attribute dependence. *Mater. Today Proc.* **2021**, *47*, 6325–6331. [\[CrossRef\]](#)
31. Vergara, J.R.; Estévez, P.A. A review of feature selection methods based on mutual information. *Neural Comput. Appl.* **2014**, *24*, 175–186. [\[CrossRef\]](#)
32. Rehman Javed, A.; Jalil, Z.; Atif Moqurrah, S.; Abbas, S.; Liu, X. Ensemble Adaboost classifier for accurate and fast detection of botnet attacks in connected vehicles. *Trans. Emerg. Telecommun. Technol.* **2022**, *33*, e4088. [\[CrossRef\]](#)
33. Syarif, I.; Zaluska, E.; Prugel-Bennett, A.; Wills, G. Application of Bagging, Boosting and Stacking to Intrusion Detection. In *Proceedings of the Machine Learning and Data Mining in Pattern Recognition*; Perner, P., Ed.; Springer: Berlin/Heidelberg, Germany, 2012; pp. 593–602.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.